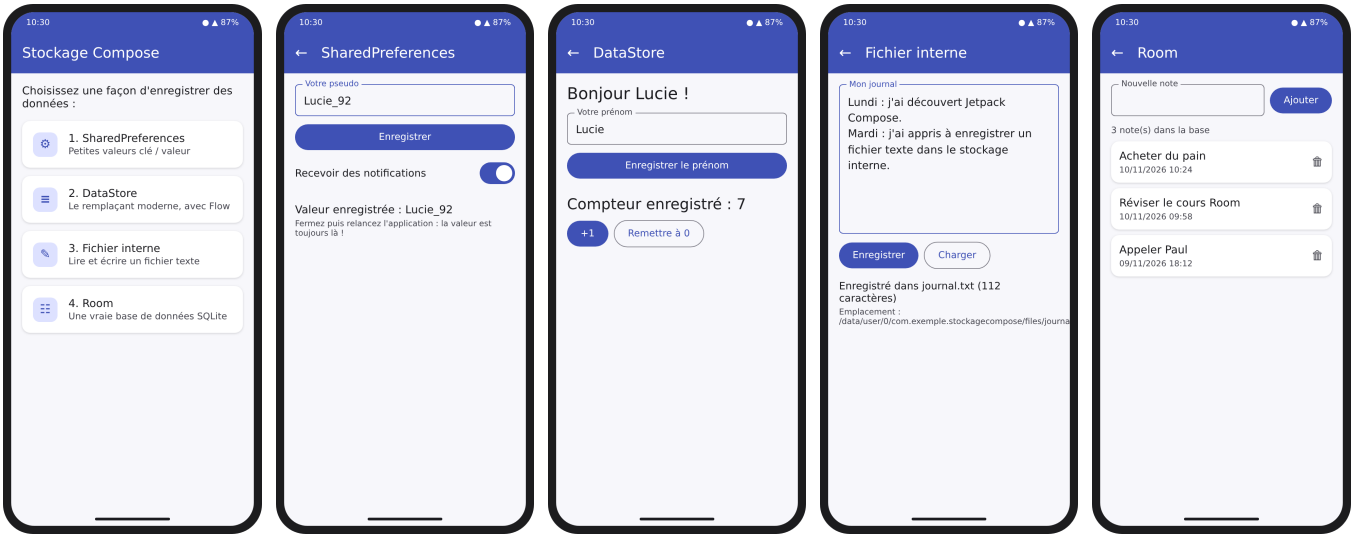
Enregistrer des données avec Jetpack Compose

Cours niveau débutant — Android / Kotlin

SharedPreferences • DataStore • Fichiers • Room



Projet d'exemple fourni : StockageCompose (Android Studio)

Sommaire

Sommaire 2

1. Pourquoi enregistrer des données ? 3

2. Le projet StockageCompose 4

3. SharedPreferences 6

4. DataStore (Preferences) 8

5. Les fichiers internes 10

6. Room : une base de données 11

7. Comment choisir ? 14

8. Exercices 14

9. Petit lexique 15

1. Pourquoi enregistrer des données ?

Quand on ferme une application Android, tout ce qui est stocké dans les variables disparaît. Si l'utilisateur a saisi son pseudo, coché une option ou écrit une note, il s'attend pourtant à les retrouver au prochain lancement. Pour cela, il faut **enregistrer** (on dit aussi **persister**) les informations sur le téléphone.

Dans ce cours, nous allons découvrir **quatre solutions**, de la plus simple à la plus complète, à travers une petite application Jetpack Compose nommée **StockageCompose**. Chaque solution a son propre écran.

Solution	Pour quoi faire ?	Exemple	Difficulté
SharedPreferences	Quelques petites valeurs (texte, nombre, booléen)	Pseudo, option activée	★☆☆
DataStore	Pareil, mais en version moderne et asynchrone	Prénom, compteur	★★☆
Fichier interne	Du texte libre ou des données brutes	Journal, export	★☆☆
Room	Beaucoup de données structurées, des listes	Liste de notes, contacts	★★★

Rappel : état Compose ≠ données enregistrées

En Compose, `remember { mutableStateOf(...) }` garde une valeur *tant que l'écran est affiché*. Ce n'est pas un enregistrement : si l'application est fermée, la valeur est perdue. Les quatre solutions de ce cours écrivent, elles, dans la **mémoire de stockage** du téléphone.

> **À retenir**
> État Compose = mémoire vive (temporaire). Stockage = mémoire du téléphone (permanent).

Prérequis

- Android Studio installé (version récente : Koala ou plus).
- Connaître les bases de Kotlin (variables, fonctions, classes).
- Savoir ce qu'est une fonction `@Composable`.

2. Le projet StockageCompose

Ouvrez le dossier **StockageCompose** fourni dans le zip avec Android Studio (*File › Open*), attendez la synchronisation Gradle, puis lancez l'application sur un émulateur ou un téléphone avec le bouton ▶.

Organisation des fichiers

```
StockageCompose/
├── settings.gradle.kts           ← nom du projet, dépôts
├── build.gradle.kts              ← versions des plugins
└── app/
    ├── build.gradle.kts          ← dépendances (DataStore, Room...)
    └── src/main/
        ├── AndroidManifest.xml
        └── java/com/exemple/stockagecompose/
            ├── MainActivity.kt           ← navigation entre écrans
            ├── data/
            │   ├── ReglagesDataStore.kt  ← (2) DataStore
            │   ├── Note.kt               ← (4) Room : la table
            │   ├── NoteDao.kt            ← (4) Room : les requêtes
            │   └── AppDatabase.kt        ← (4) Room : la base
            └── ui/
                ├── Commun.kt             ← barre du haut commune
                ├── AccueilScreen.kt
                ├── PreferencesScreen.kt  ← (1) SharedPreferences
                ├── DataStoreScreen.kt    ← (2) DataStore
                ├── FichierScreen.kt      ← (3) Fichier interne
                ├── RoomScreen.kt         ← (4) Room
                └── theme/Theme.kt
```

Les dépendances

SharedPreferences et les fichiers sont inclus dans Android : rien à ajouter. DataStore et Room sont des bibliothèques **Jetpack** qu'il faut déclarer dans `app/build.gradle.kts` :

app/build.gradle.kts (extrait)

```
plugins {
    // ...
    id("com.google.devtools.ksp") // génère le code de Room
}
dependencies {
    implementation("androidx.datastore:datastore-preferences:1.1.1")

    implementation("androidx.room:room-runtime:2.6.1")
    implementation("androidx.room:room-ktx:2.6.1")
    ksp("androidx.room:room-compiler:2.6.1")
}
```

L'écran d'accueil et la navigation



Écran d'accueil

L'écran d'accueil affiche une **carte** (`Card`) par solution de stockage.

Un clic sur une carte appelle `navController.navigate("route")` pour ouvrir l'écran correspondant.

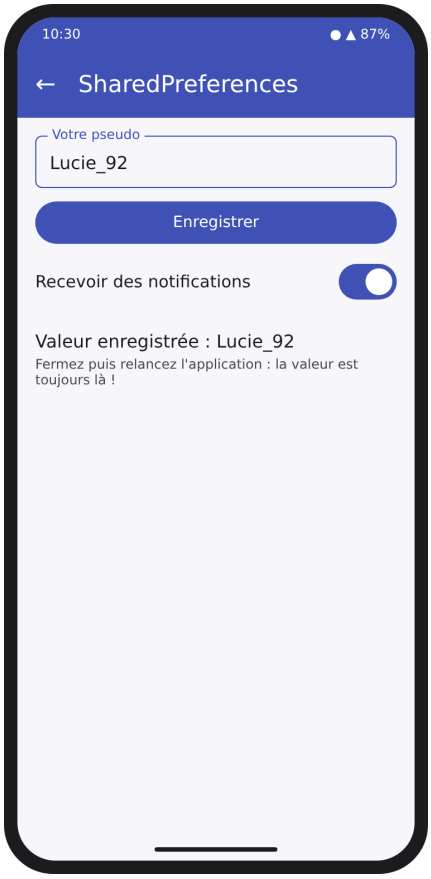
Toutes les routes sont déclarées dans `MainActivity.kt` grâce à un `NavHost` :

MainActivity.kt (simplifié)

```
@Composable
fun AppNavigation() {
    val navController = rememberNavController()
    NavHost(navController, startDestination = "accueil") {
        composable("accueil") {
            AccueilScreen(onNavigate = { route -> navController.navigate(route) })
        }
        composable("preferences") {
            PreferencesScreen(onBack = { navController.popBackStack() })
        }
        composable("datastore") { /* ... */ }
        composable("fichier")   { /* ... */ }
        composable("room")      { /* ... */ }
    }
}
```

3. SharedPreferences

SharedPreferences est la solution historique d'Android. Elle enregistre des paires **clé → valeur** dans un petit fichier XML privé à l'application. Types acceptés : `String`, `Int`, `Long`, `Float`, `Boolean` et `Set<String>`.



Écran SharedPreferences

Ce que fait l'écran :

- On saisit un pseudo puis on appuie sur **Enregistrer**.
- L'interrupteur « notifications » est sauvegardé dès qu'on le touche.
- Fermez l'application, relancez-la : les valeurs sont toujours là.

Les 3 étapes :

- **Ouvrir** les préférences avec `getSharedPreferences(nom, MODE_PRIVATE)`.
- **Lire** avec `getString(clé, défaut)`, `getBoolean(...)`…
- **Écrire** avec `edit { putString(clé, valeur) }`.

ui/PreferencesScreen.kt (simplifié)

```
val context = LocalContext.current
// 1) Ouvrir le "fichier" de préférences nommé "mes_reglages"
val prefs = remember {
    context.getSharedPreferences("mes_reglages", Context.MODE_PRIVATE)
}

// 2) Lire la valeur déjà enregistrée (ou "" par défaut)
var pseudo by remember { mutableStateOf(prefs.getString("pseudo", "") ?: "") }

// 3) Écrire la valeur quand on clique sur le bouton
Button(onClick = { prefs.edit { putString("pseudo", pseudo) } }) {
    Text("Enregistrer")
}

// Un Switch sauvegardé immédiatement
Switch(checked = notifications, onCheckedChange = {
    notifications = it
    prefs.edit { putBoolean("notifications", it) }
})
```

À retenir
Simple et rapide à mettre en place. Mais la lecture se fait sur le fil principal et Google recommande désormais **DataStore** pour les nouveaux projets.

4. DataStore (Preferences)

DataStore est le remplaçant moderne de SharedPreferences. Il stocke aussi des paires clé → valeur, mais :

- la lecture renvoie un **Flow** : l'écran se met à jour automatiquement quand la valeur change ;
- l'écriture est une fonction `suspend` : elle ne bloque jamais l'interface ;
- les clés sont **typées** (`stringPreferencesKey`, `intPreferencesKey`…), ce qui évite les erreurs.

Étape 1 : créer le DataStore et les fonctions d'accès

data/ReglagesDataStore.kt

```
// Un seul DataStore pour toute l'application (déclaré au niveau du fichier)
val Context.dataStore: DataStore<Preferences> by preferencesDataStore(name = "reglages")

/**
 * Petite classe qui regroupe la lecture / écriture dans DataStore.
 */
class ReglagesDataStore(private val context: Context) {

    companion object {
        val CLE_PRENOM = stringPreferencesKey("prenom")
        val CLE_COMPTEUR = intPreferencesKey("compteur")
    }

    // Lecture : un Flow qui émet une nouvelle valeur à chaque modification
    val prenom: Flow<String> = context.dataStore.data.map { it[CLE_PRENOM] ?: "" }
    val compteur: Flow<Int> = context.dataStore.data.map { it[CLE_COMPTEUR] ?: 0 }

    // Écriture : fonctions "suspend" à appeler dans une coroutine
    suspend fun enregistrerPrenom(valeur: String) {
        context.dataStore.edit { it[CLE_PRENOM] = valeur }
    }

    suspend fun incrementer() {
        context.dataStore.edit { it[CLE_COMPTEUR] = (it[CLE_COMPTEUR] ?: 0) + 1 }
    }
    // ...
}
```

Étape 2 : utiliser DataStore dans l'écran



Écran DataStore

`collectAsState()` transforme le Flow en état Compose : dès que le prénom ou le compteur change dans DataStore, le texte à l'écran est redessiné.

Comme `enregistrerPrenom()` est une fonction `suspend`, on l'appelle dans une coroutine créée par `rememberCoroutineScope()` :

ui/DataStoreScreen.kt (extrait)

```
val scope = rememberCoroutineScope()

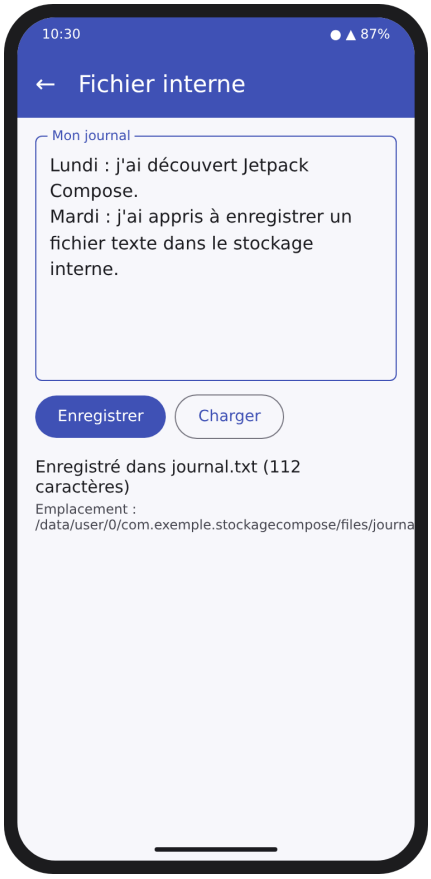
val prenomEnregistre by reglages.prenom.collectAsState(initial = "")
val compteur by reglages.compteur.collectAsState(initial = 0)

Button(onClick = { scope.launch { reglages.enregistrerPrenom(saisie) } }) {
    Text("Enregistrer le prénom")
}
Button(onClick = { scope.launch { reglages.incrementer() } }) { Text("+1") }
```

À retenir
Lire = `Flow` + `collectAsState()`. Écrire = `scope.launch { dataStore.edit { ... } }`. On déclare **un seul** DataStore par fichier dans toute l'application.

5. Les fichiers internes

Chaque application possède un dossier privé : `context.filesDir`. Les autres applications n'y ont pas accès et **aucune permission** n'est nécessaire. C'est idéal pour enregistrer du texte libre, un export JSON ou une image.



Écran Fichier interne

Enregistrer écrit le contenu du champ dans `journal.txt`.

Charger relit le fichier et remplit le champ.

On utilise deux fonctions Kotlin toutes simples :

- `openFileOutput()` ouvre un flux d'écriture ;
- `File(...).readText()` lit tout le contenu.

`MODE_PRIVATE` remplace le fichier à chaque écriture. Pour ajouter à la fin, utilisez `MODE_APPEND`.

ui/FichierScreen.kt (extrait)

```
private const val NOM_FICHIER = "journal.txt"

// Écrit le texte dans le stockage interne de l'application
fun ecrireFichier(context: Context, texte: String) {
    context.openFileOutput(NOM_FICHIER, Context.MODE_PRIVATE).use { flux ->
        flux.write(texte.toByteArray())
    }
}

// Lit le fichier s'il existe, sinon renvoie une chaîne vide
fun lireFichier(context: Context): String {
    val fichier = File(context.filesDir, NOM_FICHIER)
    return if (fichier.exists()) fichier.readText() else ""
}
```

Attention
Le fichier est supprimé si l'utilisateur désinstalle l'application ou vide ses données. Pour un gros fichier, faites l'écriture dans une coroutine avec `Dispatchers.IO`.

6. Room : une base de données

Quand on a **beaucoup** d'éléments du même type (des notes, des contacts, des dépenses...), on utilise une base de données **SQLite**. **Room** est la bibliothèque Jetpack qui rend SQLite simple à utiliser en Kotlin. Elle repose sur **trois briques** :

Brique	Annotation	Rôle
Entity	@Entity	Décrit une **table** : chaque propriété devient une colonne.
DAO	@Dao	Liste les **opérations** : lire, ajouter, supprimer...
Database	@Database	La **base** elle-même, qui relie entités et DAO.

Brique 1 : l'entité Note

data/Note.kt

```
@Entity(tableName = "notes")
data class Note(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val titre: String,
    val dateCreation: Long = System.currentTimeMillis()
)
```

Brique 2 : le DAO

data/NoteDao.kt

```
@Dao
interface NoteDao {

    @Query("SELECT * FROM notes ORDER BY dateCreation DESC")
    fun toutesLesNotes(): Flow<List<Note>>

    @Insert
    suspend fun ajouter(note: Note)

    @Delete
    suspend fun supprimer(note: Note)
}
```

Une requête qui renvoie un `Flow` se met à jour automatiquement à chaque ajout ou suppression. Les fonctions qui modifient la base sont `suspend`.

Brique 3 : la base de données

data/AppDatabase.kt

```
@Database(entities = [Note::class], version = 1)
abstract class AppDatabase : RoomDatabase() {

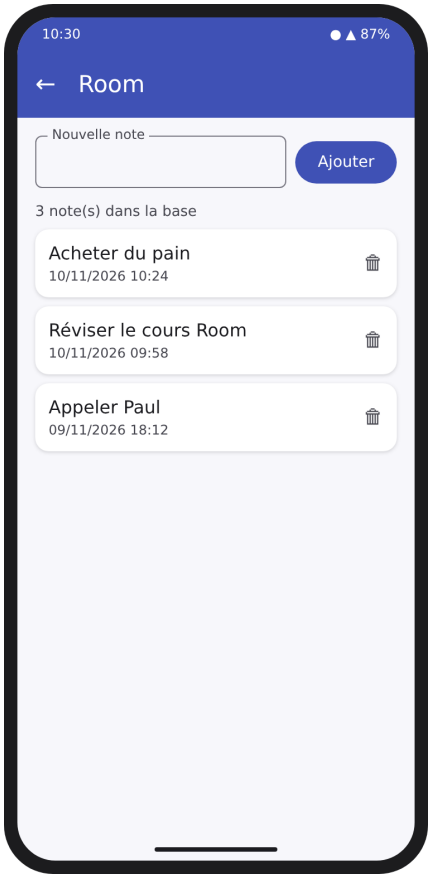
    abstract fun noteDao(): NoteDao

    companion object {
        @Volatile
        private var instance: AppDatabase? = null

        // On crée la base une seule fois (singleton)
        fun get(context: Context): AppDatabase =
            instance ?: synchronized(this) {
                instance ?: Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "mes_notes.db"
                ).build().also { instance = it }
            }
    }
}
```

Le **singleton** (une seule instance) évite d'ouvrir la base plusieurs fois, ce qui coûte cher.

L'écran Room



Écran Room

On tape un titre puis **Ajouter** : une ligne est insérée dans la table `notes`.

La liste (`LazyColumn`) observe `toutesLesNotes()` : elle se met à jour toute seule.

L'icône poubelle supprime la note de la base.

Fermez l'application : les notes sont conservées.

ui/RoomScreen.kt (simplifié)

```
val dao = remember { AppDatabase.get(context).noteDao() }
val scope = rememberCoroutineScope()
val notes by dao.toutesLesNotes().collectAsState(initial = emptyList())

Button(onClick = { scope.launch { dao.ajouter(Note(titre = saisie)) } }) {
    Text("Ajouter")
}
LazyColumn {
    items(notes, key = { it.id }) { note ->
        Text(note.titre)
        IconButton(onClick = { scope.launch { dao.supprimer(note) } }) { /* ... */ }
    }
}
```

À retenir

Entity = table, DAO = requêtes, Database = la base. Si vous modifiez une Entity, augmentez le numéro `version` de la base (et prévoyez une migration).

7. Comment choisir ?

Question	Réponse
Je stocke 1 à 20 réglages simples (thème, pseudo, option) ?	**DataStore** (ou SharedPreferences dans un projet existant)
Je veux que l'écran se mette à jour tout seul ?	**DataStore** ou **Room** (ils fournissent un Flow)
J'enregistre un texte libre, un JSON, une image ?	**Fichier interne**
J'ai une liste d'objets à trier, filtrer, rechercher ?	**Room**
Les données doivent être partagées entre téléphones ?	Aucune des 4 : il faut un serveur (Firebase, API...)

Bonnes pratiques

- Ne jamais faire d'écriture lente directement dans un `onClick` sans coroutine (sauf SharedPreferences avec `apply`).
- Ne pas stocker de mot de passe en clair.
- Dans une vraie application, placez l'accès aux données dans un **ViewModel** plutôt que directement dans le Composable (ici on simplifie pour apprendre).
- Donnez des noms de clés clairs et constants (`companion object`).

8. Exercices

Exercice 1 — Thème sombre
Ajoutez un interrupteur « Mode sombre » enregistré avec DataStore et appliquez-le dans `StockageTheme`.

Exercice 2 — Journal daté
Dans l'écran Fichier, utilisez `MODE_APPEND` pour ajouter chaque entrée à la suite, précédée de la date.

Exercice 3 — Notes favorites
Ajoutez une colonne `favori: Boolean` à l'entité Note (pensez à la version !) et une étoile cliquable.

Exercice 4 — Recherche
Ajoutez au DAO une requête `WHERE titre LIKE '%' || :texte || '%'` et un champ de recherche.

Exercice 5 — Compteur de lancements
Comptez combien de fois l'application a été ouverte avec SharedPreferences et affichez-le sur l'accueil.

9. Petit lexique

Mot	Définition
Persister	Enregistrer une donnée pour qu'elle survive à la fermeture de l'application.
Clé / valeur	Une étiquette (la clé, ex. "pseudo") associée à une donnée (la valeur, ex. "Lucie").
Flow	Un flux Kotlin qui émet une nouvelle valeur chaque fois que la donnée change.
suspend	Une fonction qui peut prendre du temps et doit être appelée depuis une coroutine.
Coroutine	Une tâche légère qui s'exécute sans bloquer l'interface (ex. `scope.launch { }`).
SQLite	Le moteur de base de données intégré à Android, utilisé par Room.
KSP	Outil qui génère automatiquement du code à la compilation (utilisé par Room).